

## CONTENTS

|                                                           |   |
|-----------------------------------------------------------|---|
| 1. PROJECT INFORMATION                                    | 2 |
| 1.1. Project Title                                        | 2 |
| 1.2. Team Information                                     | 2 |
| 2. INTRODUCTION (5 pts)                                   | 3 |
| 2.1. Motivation/Background                                | 3 |
| 2.2. State of the Art / Innovation                        | 3 |
| 2.3. Project Goals                                        | 3 |
| 2.4. Assumptions                                          | 3 |
| 3. PLANNED SYSTEM ARCHITECTURE (20 pts)                   | 4 |
| 3.1. High-Level Diagram (10 pts)                          | 4 |
| 3.2. User Interaction and Design (10 pts)                 | 5 |
| 4. SYSTEM MODELS (10 pts)                                 | 6 |
| 5. SOFTWARE TOOLS/PACKAGES UTILIZED SO FAR (5 pts)        | 6 |
| 6. PROJECT IMPLEMENTATION (20 PTS)                        | 7 |
| 6.1. Tasks/Milestones Completed & Issue Tracking (10 pts) | 7 |
| 6.2. Challenges/Roadblocks (5 pts)                        | 7 |
| 6.3. Tasks to be Completed next quarter (5 pts)           | 8 |
| 7. WEAK POINTS / FUTURE WORK (5 PTS)                      | 8 |
| 8. DELIVERABLES (35 PTS)                                  | 8 |
| 8.1. Progress Demo Video (20 Pts)                         | 8 |
| 8.2. Source Code & Repository Health (15 pts)             | 8 |

## 1. PROJECT INFORMATION

### ○ Project Title

ProtoPilot - Turning Product Vision into Working Prototypes

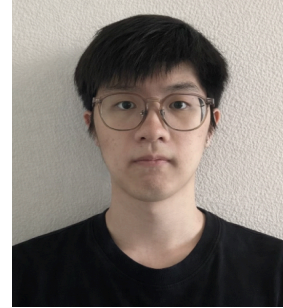
### ○ Team Information

|                                                                                                                             |                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <i>Team Name:</i><br><i>Team # on Canvas:</i>                                                                               | <b>Team OXYZen</b><br><b>I.1</b>                                                      |
| <b>Team member 1</b><br><i>(Last Name, name; student ID; UCI email; picture):</i><br>Dabir, Omkar; 81794560; odabir@uci.edu |    |
| <b>Team member 2</b><br><i>(Last Name, name; student ID; UCI email, picture):</i><br>Jiang, Xin; 50846523; jiangx15@uci.edu |  |
| <b>Team member 3</b><br><i>(Last Name, name; student ID; UCI email, picture):</i><br>Yiniu Han; 13234789; yiniuh@uci.edu    |  |

**Team member 4**

(Last Name, name: student ID: UCI email, picture):

Zhihao Wang; 28851744; zhihw15@uci.edu



## 2. INTRODUCTION (5 pts)

- Motivation/Background

In the early stage of product development, implementing the product vision or idea quickly and accurately into a runnable and testable prototype has always been a common bottleneck. Prototype development often takes a long time and consumes a large amount of engineering resources. Also, during the continuous clarification and iteration of requirements, it is easy for there to be a deviation between the understanding of the product owner and the final implementation. This will directly slow down the verification speed, increase the cost of trial and error, and weaken the ability of innovation and iteration.

With the development of large language models (LLM) and multi-agent (Agentic AI) capabilities, using "human-in-the-loop + multi-role AI agent" to cover requirements clarification, solution generation, code implementation and test verification is becoming a feasible path to accelerate prototype development. This project focuses on building an AI-driven platform that enables non-technical users to transform product ideas into functional web-based prototypes through a guided, human-in-the-loop workflow. The system will orchestrate multiple specialized AI components to generate both a working proof-of-concept application and structured development artifacts such as requirements, user flows, and technical outlines. The goal is to reduce friction and improve alignment during ideation and validation, enabling faster iteration and clearer communication—rather than delivering a production-ready system.

- State of the Art / Innovation

In practice, early-stage product prototyping is commonly carried out through manual and collaboration-heavy processes. Product ideas are usually documented using written specifications, presentation slides, or visual mockups created with design tools. Requirement clarification often happens through meetings and discussions between product owners, designers, and engineers. Based on these inputs, developers implement a proof-of-concept using standard web frameworks or internal project templates.

While this workflow is widely used, it can be time-consuming, especially when requirements are unclear or evolve during development. In such cases, multiple iterations are often needed before a prototype accurately reflects the original idea. Low-code and no-code platforms are sometimes used to speed up development for

simple applications, but they typically support limited functionality and are not well suited for generating artifacts such as detailed requirements or development plans.

More recently, large language model-based tools have been introduced to assist with software development tasks, mainly focusing on code generation or developer productivity. However, these tools are generally designed for technical users and do not directly address the early-stage process of requirement clarification and idea validation for non-technical stakeholders.

## ○ Project Goals

This project will deliver an AI-based platform that helps non-technical users turn their ideas into working web application prototypes. The system will generate a full-stack proof-of-concept using Java Spring Boot and Angular, and also create useful development artifacts such as specifications, development plans, and task breakdowns that follow SpecKit and BMAD methods.

The main outcomes are faster requirement gathering, quicker prototyping, and better communication between product owners and developers. Instead of waiting months for feedback, users can see and test their ideas early. The generated artifacts can be reused by engineers to continue real product development, which reduces repeated work and misunderstanding. The project also shows how multi-agent AI systems can be used in real software engineering workflows.

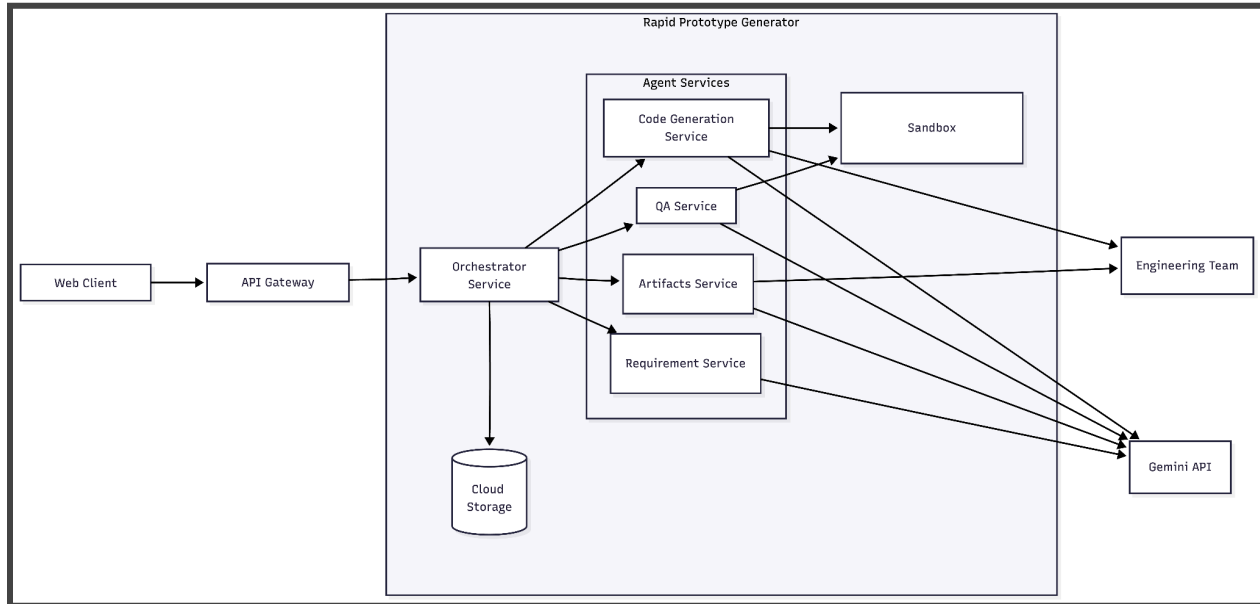
In the final demo, we will show the full process: a user enters an idea, the system asks questions to clarify requirements, generates a working prototype, runs basic tests, and outputs the development documents. This will demonstrate the complete flow from idea to deployable PoC.

## ○ Assumptions

- Users are able to describe product ideas in natural language and are willing to participate in clarification steps within a human-in-the-loop workflow.
- During the demo and evaluation phase, the number of concurrent users will remain limited.
- The generated proof-of-concept (PoC) applications target Java and Angular as output technologies.
- External AI services (Gemini API or others) are available and respond within acceptable latency during demonstration and evaluation.

### 3. PLANNED SYSTEM ARCHITECTURE (20 pts)

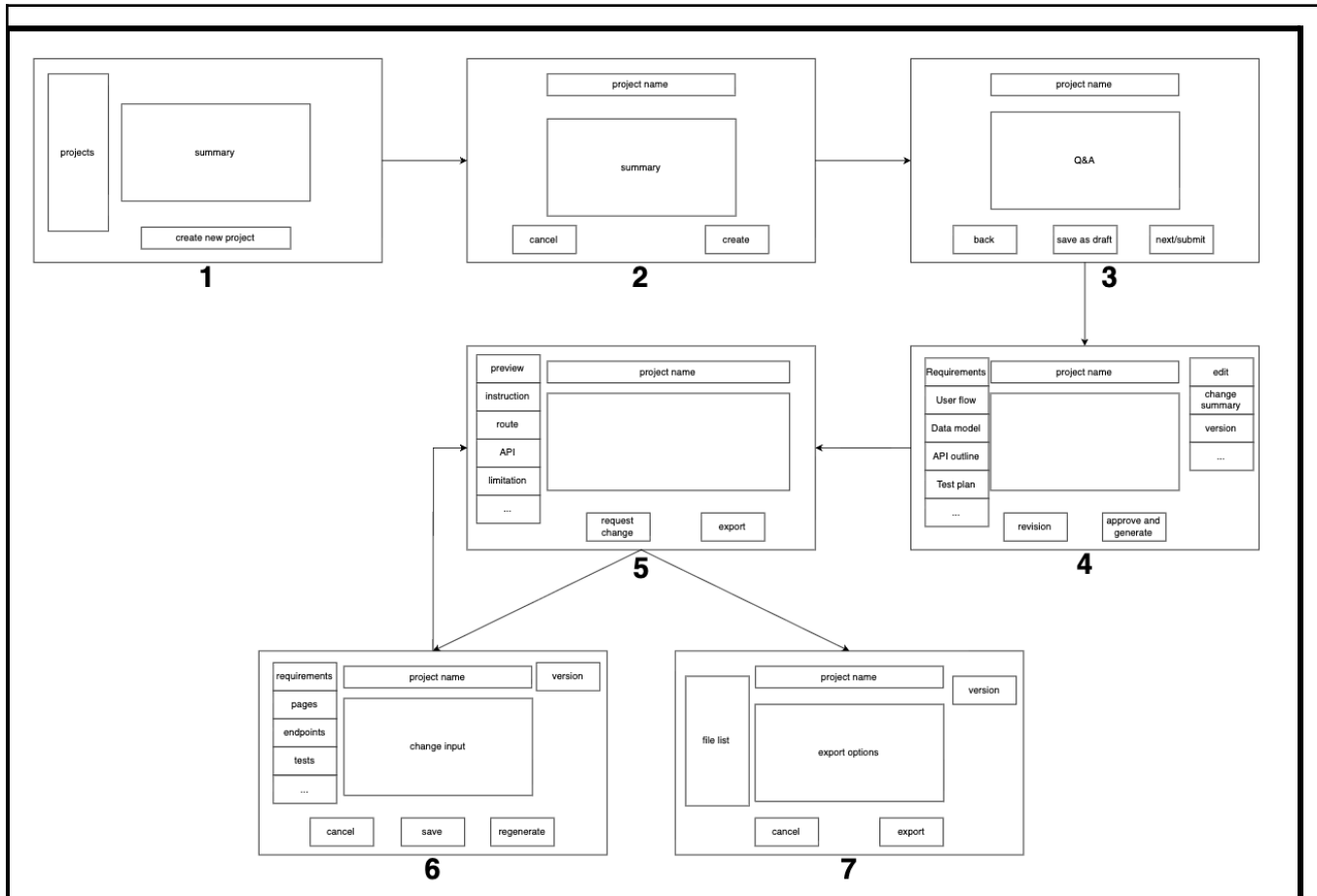
- High-Level Description & Diagram (10 pts)



**Figure 1: High Level Diagram**

A user interacts through a web client, and their request goes through an API Gateway to an Orchestrator Service, which acts as the brain of the system. The orchestrator coordinates several agent services: a Requirement Service to understand and refine the user's needs, a Code Generation Service to generate application code, a QA Service to validate and test it, and an Artifacts Service to package and manage the generated outputs. These services can call the Gemini API to generate content using AI. The Code Generation Service can also send code to a sandbox environment to safely build and run it. All important data and outputs are stored in cloud storage. Finally, the generated artifacts and results can be shared with the engineering team for review, improvement, or production deployment.

- User Interaction and Design (10 pts)



**Figure 2: User Interaction Diagram**

As shown in Figure 2.1(Dashboard), the dashboard is the entry point of the system. It provides a simple overview of existing projects and their current status. Users can resume work on a selected project or start a new one using the Create New Project.

As shown in Figure 2.2 (Create Project), the create-project interface collects minimal metadata required to initialize a workspace, including project name and a short summary. Users can either cancel the action or confirm creation. Upon creation, the system navigates the user directly into the guided intake workflow.

As shown in Figure 2.3(Guided Q&A Wizard), the wizard is a step-by-step interface for requirement gathering. Users answer questions about the product goal, target users, key features, constraints, and success criteria. The wizard supports Back navigation and Save as Draft. When the system needs to infer missing details, it can surface assumptions for user confirmation before submission.

As shown in Figure 2.4(Spec Review Workspace), after the wizard submission, the system generates structured artifacts and presents them in a review workspace. The left panel provides navigation across artifacts. Users can edit and refine items, review the change summary, and track versions. Two main actions are provided: Revision (request updated artifacts) and Approve and Generate (freeze the spec version and initiate prototype generation).

As shown in Figure 2.5(Prototype Preview and Validation), once generation completes, the preview interface helps users validate the proof-of-concept. A side navigation organizes key validation views. From this page,

users can either continue iterating via Request Change or proceed to Export.

As shown in Figure 2.6(Change Request and Impact Confirmation), the change request interface is used to iterate on the prototype in a controlled manner. Users provide a change description (free-text or structured) and review an impact scope. Users may cancel, save the change request, or regenerate, which produces a new version while preserving prior versions for traceability.

As shown in Figure 2.7(Export and Handoff), the export interface packages the generated prototype and artifacts into a handoff bundle. Users can review a file list, select export options and then export the package.

## 4. SYSTEM MODELS (10 pts)

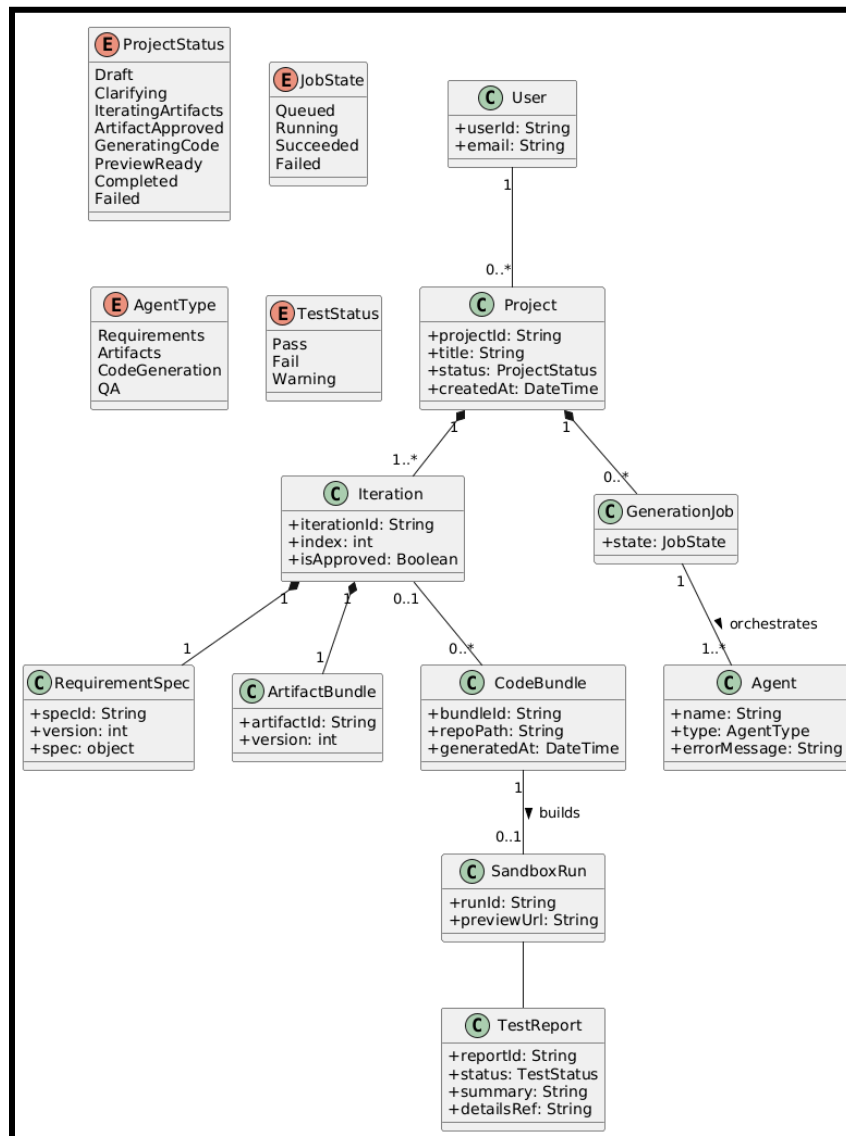


Figure 5: Core Class Diagram

**User and Project Relationship:**

A User can own multiple Projects. Each Project represents a complete prototype generation workflow and progresses through states defined by *ProjectStatus*.

**Iterative Refinement Model:**

Each Project contains one or more Iterations. An Iteration represents a single revision cycle, including a *RequirementSpec* and a corresponding *ArtifactBundle*. This design supports multiple rounds of clarification and artifact regeneration.

**Code Generation:**

An Iteration includes an *isApproved* flag. Only approved iterations can produce a *CodeBundle*, ensuring that code generation occurs after artifact confirmation.

**Execution and Orchestration:**

*GenerationJob* models background execution tasks (e.g., specification processing, code generation). Each job has a lifecycle defined by *JobState* and may orchestrate multiple *Agent* components.

**Preview and Testing:**

A *CodeBundle* may be built and executed in a *SandboxRun*, which generates a preview URL. The sandbox execution can produce a *TestReport* summarizing testing results.

## 5. SOFTWARE TOOLS/PACKAGES UTILIZED SO FAR (5 pts)

**Live Application URL** (if applicable): Local Only for now

**Deployment Platform** (or "Local Only"): Will host using Vercel before end of April

| Software Tool / Package            | Purpose / How it is used in the project                                        |
|------------------------------------|--------------------------------------------------------------------------------|
| Google ADK (Agent Development Kit) | Framework for building and running multi-turn LLM agents in the pipeline       |
| LiteLLM                            | Unified LLM proxy client used to call Gemini models via the Cotality proxy     |
| FastAPI                            | Backend web framework that exposes the /chat and /jobs REST API endpoints      |
| Angular                            | Frontend UI framework for the requirements gathering and spec review interface |



## 6. PROJECT IMPLEMENTATION (20 PTS)

### ○ Tasks/Milestones Completed & Issue Tracking (10 pts)

Issue Tracker:

<https://github.com/users/odabir33/projects/1>

<https://github.com/odabir33/ProtoPilot/issues>

| <b>Task Completed</b>                                         | <b>Issue / Ticket #</b> | <b>Team Member(s)</b>  |
|---------------------------------------------------------------|-------------------------|------------------------|
| <i>Initial project setup and Requirements Gathering Agent</i> | #30                     | Omkar Dabir            |
| <i>Set up LiteLLM proxy connection</i>                        | #31                     | Omkar Dabir            |
| <i>Centralize LLM and auth logic into core modules</i>        | #9                      | Xin Jiang              |
| <i>Expose agents via REST API</i>                             | #28                     | Xin Jiang              |
| <i>Standardize agent structure with central registry</i>      | #29                     | Xin Jiang              |
| <i>Implement Requirements Gathering Agent (refine)</i>        | #10                     | Xin Jiang              |
| <i>Implement Artifacts Generation Agent</i>                   | #11                     | Yiniu Han, Xin Jiang   |
| <i>Support different LLM for different agent</i>              | #21                     | Xin Jiang              |
| <i>Implement Codegen Agent for Spring Boot + Angular</i>      | #12                     | Xin Jiang              |
| <i>Local deployment pipeline for generated apps</i>           | #13                     | Xin Jiang              |
| <i>Fix WAF blocking LLM requests</i>                          | #14                     | Xin Jiang              |
| <i>Fix LiteLLM httpx.URL serialization bug</i>                | #18                     | Xin Jiang              |
| <i>Initial Angular frontend scaffold</i>                      | #23                     | Omkar Dabir            |
| <i>Add CORS support to API server</i>                         | #22                     | Omkar Dabir, Xin Jiang |
| <i>Requirements gathering UI</i>                              | #15                     | Omkar Dabir            |
| <i>Spec review page initial scaffold</i>                      | #32                     | Omkar Dabir            |

| <i>Task Completed</i>                                                                                                       | <i>Issue / Ticket #</i> | <i>Team Member(s)</i> |
|-----------------------------------------------------------------------------------------------------------------------------|-------------------------|-----------------------|
| <i>Spec review UI with chatbox and artifact panels</i>                                                                      | <i>#24</i>              | <i>Omkar Dabir</i>    |
| <i>Dynamic spec loading and live preview component</i>                                                                      | <i>#25</i>              | <i>Omkar Dabir</i>    |
| <i>Add global loader and Mermaid diagram rendering</i>                                                                      | <i>#26</i>              | <i>Omkar Dabir</i>    |
| <i>Codegen trigger and file preview in spec review</i>                                                                      | <i>#27</i>              | <i>Omkar Dabir</i>    |
| <i>Research on what is GitHub SpecKit and how we can use it in our project</i>                                              | <i>-</i>                | <i>Zhihao Wang</i>    |
| <i>Research about memory management for storing artefacts (adk memory service vs adk artefacts service vs own database)</i> | <i>#19</i>              | <i>Yiniu Han</i>      |

## ○ Challenges/Roadblocks (5 pts)

### 1. LLM Rate Limits and Token Constraints

One of the initial challenges we faced was dealing with rate limits and token restrictions on free-tier LLM APIs. Since our application relies heavily on multiple LLM interactions across stages such as questionnaire generation, specification synthesis, and code generation, these limitations significantly hindered development and testing. This issue was alleviated with the support of our project sponsors, who provided access to their LLM subscription via a LiteLLM proxy, enabling more reliable and scalable usage.

### 2. Validating and Refining Generated Code

Ensuring the correctness of AI-generated code has been another major challenge. The outputs from the code generation agent are not always fully accurate or bug-free. To address this, we are designing an adversarial workflow where a secondary LLM reviews and iteratively improves the generated code. In addition, we plan to introduce a QA agent that generates unit tests to validate core use cases. To further enhance robustness, we aim to provide a chat-based interface alongside the live preview, allowing users to request fixes for runtime errors dynamically.

### 3. Backend Execution and Preview Strategy

We also explored multiple approaches for enabling backend execution and preview. These include using browser-based tools like CodeSandbox for frontend-focused previews, setting up locally hosted backends with mocked APIs, and evaluating advanced sandboxed environments such as Daytona. Each approach presents trade-offs in terms of integration complexity, performance, and realism. At this stage, we are still evaluating and have not

finalized whether to proceed with CodeSandbox or Daytona, as we aim to balance feasibility with a seamless user experience.

- Tasks to be Completed this quarter (5 pts)

| Task                                                           | Reason                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Complete a working frontend prototype with live preview</i> | <i>Although we completed the main frontend scaffold and the initial spec review interface, the full live preview workflow was not finalized. Additional work is still needed to connect generated outputs to a stable and user-friendly preview experience.</i>                        |
| <i>Implement a mocked backend flow for the MVP demo</i>        | <i>A smoother end-to-end MVP demo was originally planned through a mocked backend flow. This was not fully completed because we prioritized building the actual core services and integrating the main project pipeline first.</i>                                                     |
| <i>Add a chat-based refinement interface</i>                   | <i>We planned to provide a chat-based UI to allow users to iteratively refine generated outputs. This feature was postponed in order to focus on the requirements gathering flow and artifact review workspace, which were considered higher-priority deliverables for this stage.</i> |
| <i>Deploy or host the backend for external access</i>          | <i>While the backend architecture and REST API endpoints have been implemented, the backend is still running locally and has not yet been fully deployed. This delay was mainly caused by the additional time spent evaluating feasible backend preview and execution strategies.</i>  |
| <i>Improve validation of generated backend/API code</i>        | <i>We planned to deliver more thoroughly validated generated code. However, ensuring the correctness and robustness of AI-generated backend code remains challenging, and more effort is required for testing, verification, and iterative refinement.</i>                             |
| <i>Improve export and packaging of generated artifacts</i>     | <i>Although artifact generation has already been implemented, the export and handoff process is not yet complete. More work is needed to support cleaner packaging of generated files and code bundles for downstream engineering use.</i>                                             |

## 7. WEAK POINTS / FUTURE WORK (5 PTS)

*One weakness of the current project is that some parts of the workflow still rely on incomplete or partially mocked integrations. While this was sufficient for early prototyping and interface development, it means that the system does not yet provide a fully seamless end-to-end experience from requirement clarification to runnable preview. In future work, we plan to replace the remaining mocked flows with actual API integrations so that frontend interactions, generated artifacts, and prototype previews are all supported by the live backend services.*

*Another weakness is the lack of a fully deployed backend and persistent storage layer. At the current stage, the system mainly runs in a local environment, and support for long-term session persistence, version history, and artifact storage is still limited. This makes it harder for users to revisit projects, compare iterations, and manage generated outputs across sessions. As future work, we plan to deploy the backend in a stable hosted environment and introduce a database-backed persistence layer for projects, sessions, and artifacts.*

*In addition, the quality assurance process for AI-generated code still has room for improvement. Although the system can already generate application code and supporting artifacts, the generated outputs are not always reliable enough to be used without further validation. Future work will include stronger automated testing, adversarial review by another LLM, and better integration of QA feedback into the regeneration workflow. These improvements will make the platform more robust and more useful for real software engineering scenarios.*

## 8. DELIVERABLES (35 PTS)

- Progress Demo Video (20 Pts)

**Demo Link:**

[https://drive.google.com/drive/folders/18ltKpqj0VTusTmuH1gWiZLR-MEe2yAhU?usp=drive\\_link](https://drive.google.com/drive/folders/18ltKpqj0VTusTmuH1gWiZLR-MEe2yAhU?usp=drive_link)

- Source Code & Repository Health (15 pts)

**Repository Link:**

<https://github.com/odabir33/ProtoPilot>